Tensor
BEYOND DIMENSIONS

Model Forge

Episode 1 Part 1

Basics of ML model and Gradient Descent

By : -Ishwor Raj Pokharel
Coordinator, TensorIOE
IOE Thapathali Campus

What is machine learning?

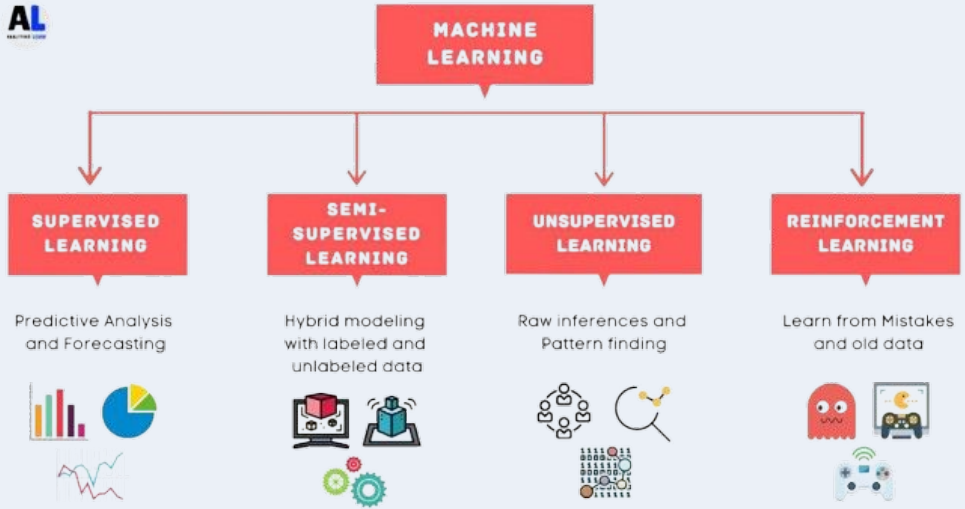
An ML system :

1) Learns Patterns from data
2) Performs tasks (prediction, recognition, classification)
3) Is not explicitly programmed for each task

Instead of writing rules manually, we let the machine figure out the rules by looking at data.

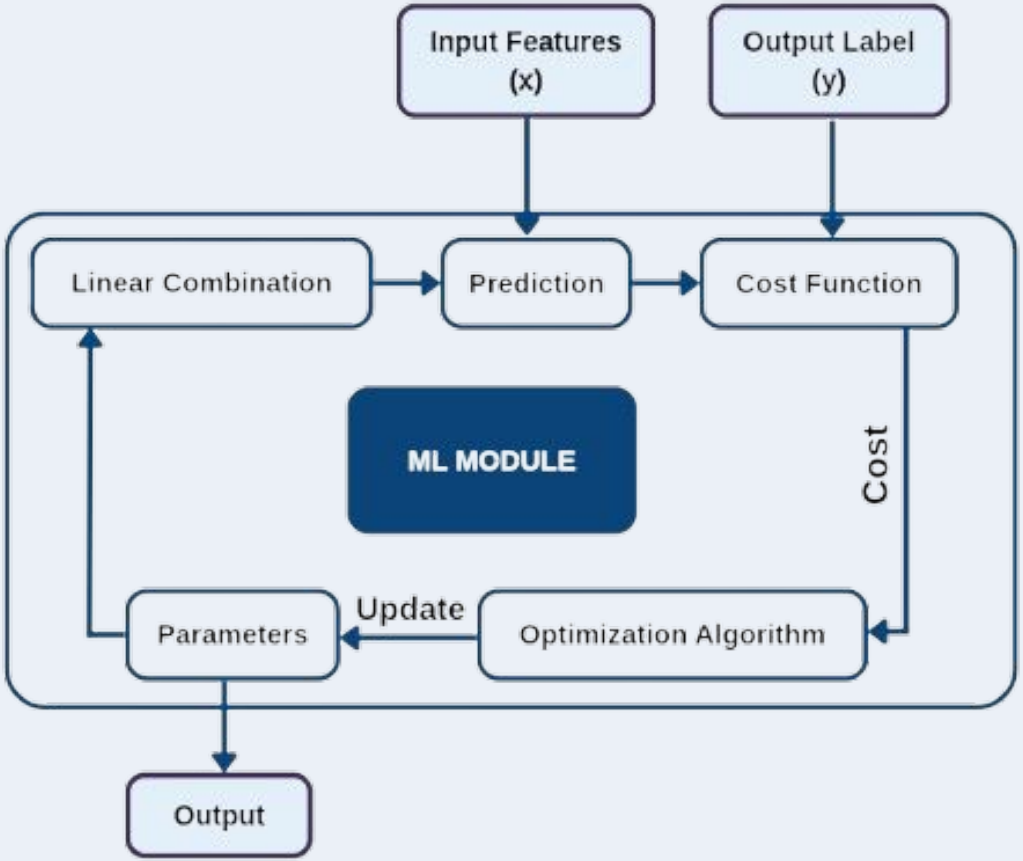
"A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience. E"

– Tom Mitchell



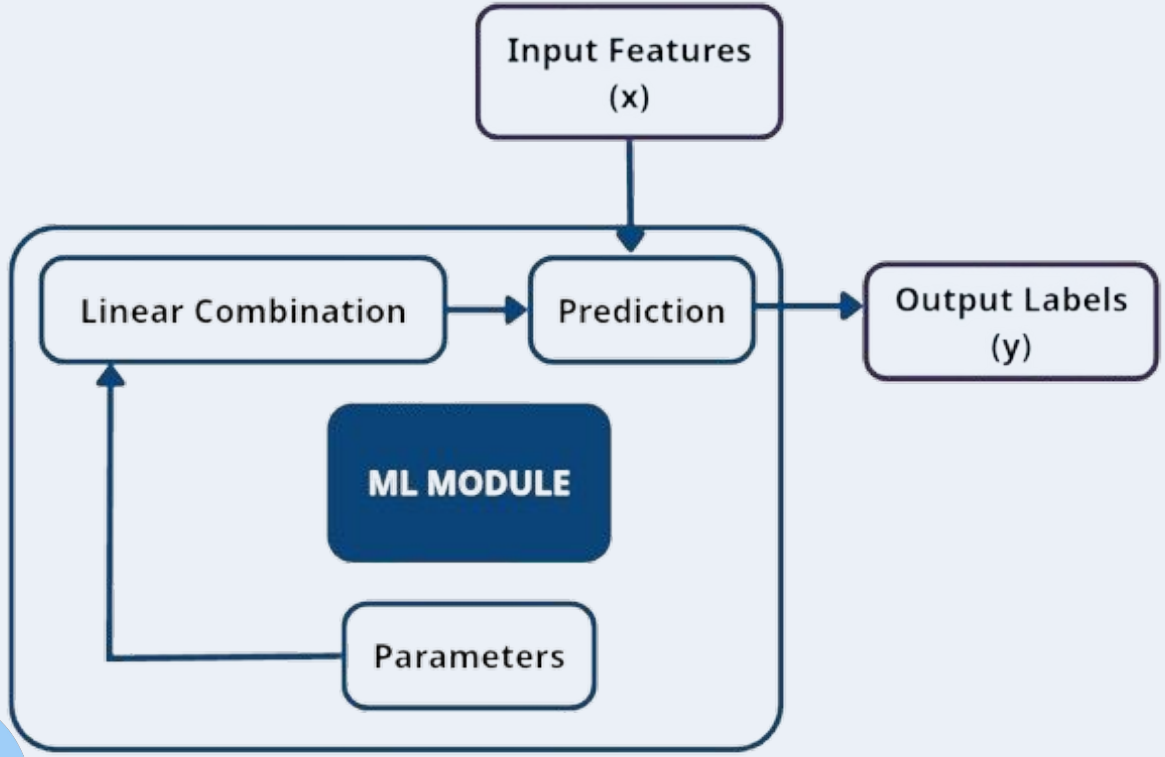
Tensor
BEYOND DIMENSIONS

Structure of ML model in Training Phase



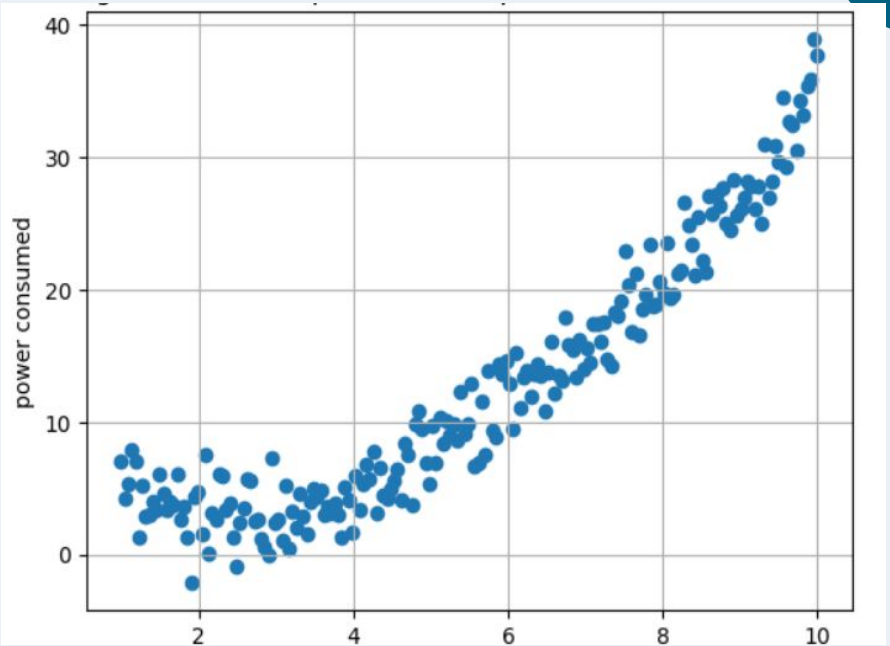
Tensor
BEYOND DIMENSIONS

Structure of trained ML Model



Tensor
BEYOND DIMENSIONS

Data



In machine learning, data are the facts, measurements, or observations used to train, test, and evaluate models.

They’re the raw material from which models learn patterns, relationships, and predictions.

Tensor
BEYOND DIMENSIONS

Different types of data used by ML models

1. **Numerical Data (Quantitative):**
   Most commonly available and used in ML.
   These data can be used in statistical and arithmetic calculations. They can be continuous, e.g., Temperature, pressure, income, etc., or discrete, e.g., number of children, etc.
2. **Categorical Data (Qualitative):**
   They represent categories or labels and can't be used in mathematical calculations. They can be nominal (no inherent order, e.g., colors) or ordinal (has inherent order, e.g., rating).
3. **Text Data:**
   Raw text or processed into tokens or embeddings. These data are used by complex models such as Transformers.
   Example: product reviews, tweets, news articles
4. **Image Data:**
   Typically represented as arrays of pixel values
   Example: handwritten digits, medical scans, etc.
   We need complex models like CNN to process such data. Commonly they are used in object detection and computer vision applications.
5. **Audio and Video Data:**
   Audio data is represented as waveforms or spectrograms. E.g., voice recording, music, etc.
   These data are used in speech recognition, music classification, etc.
   Video data are represented as a series of pictures mixed with audio and sometimes even with text.
6. **Timeseries, Graph, etc.**

Understanding the data

player	nation	position	team	age	90s	touches	deffensive_touches	middle_touches	attacking_touches	attempted_take_ons	successful_take_ons
Max Aarons	ENG	DF	Bournemouth	25.0	1.0	73	19	40	15	2	0
Joshua Acheampong	ENG	DF	Chelsea	19.0	1.9	137	48	81	8	2	1
Tyler Adams	USA	MF	Bournemouth	26.0	20.8	1279	338	741	214	17	3
Tosin Adarabioyo	ENG	DF	Chelsea	27.0	14.7	1255	596	633	27	5	4
Simon Adingra	CIV	FW,MF	Brighton	23.0	11.7	529	66	168	302	49	21
Emmanuel Agbadou	CIV	DF	Wolves	27.0	14.7	1122	616	487	27	12	9
Asher Agbinone	ENG	MF	Crystal Palace	19.0	0.1	8	0	2	6	2	0
Ola Aina	NGA	DF	Nott'ham Forest	28.0	32.4	1652	597	698	387	69	32
Rayan AÃ¯t-Nouri	ALG	DF	Wolves	23.0	33.8	2092	560	900	661	137	63
Kristoffer Ajer	NOR	DF	Brentford	27.0	15.9	788	264	364	166	16	7

Player	Nation	Position	Team	Age	Weekly	Annual
Kevin De Bruyne	BEL	MF,FW	Manchester City	33	488501	25402046
Erling Haaland	NOR	FW	Manchester City	24	457970	23814418
Casemiro	BRA	MF	Manchester Utd	32	427438	22226790
Mohamed Salah	EGY	FW	Liverpool	32	427438	22226790
Bruno Fernandes	POR	MF	Manchester Utd	29	366376	19051534
Marcus Rashford	ENG	FW,MF	Manchester Utd	26	366376	19051534
Bernardo Silva	POR	MF,FW	Manchester City	29	366376	19051534
Jack Grealish	ENG	FW,MF	Manchester City	28	366376	19051534
Omar Marmoush	EGY	FW,MF	Manchester City	25	396024	20593224
Kai Havertz	GER	FW,MF	Arsenal	25	341951	17781432

Input Features: Input features are the variables or attributes used to make predictions or classifications in a model. They are the measurable properties of data that the algorithm analyzes to identify patterns and relationships.

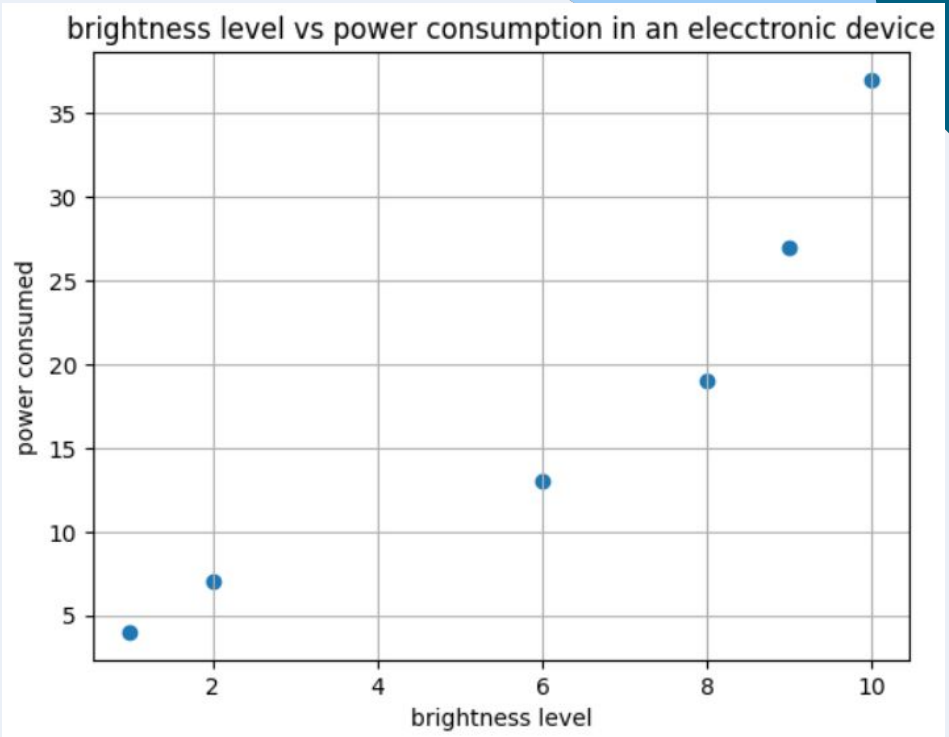
Output Labels: Output labels (also called target variables or dependent variables) are the values that the model aims to predict. They represent the expected outcome based on the input features.

Our Data

Brightness level	Power Consumed
1	4
2	7
6	13
8	19
9	27
10	37

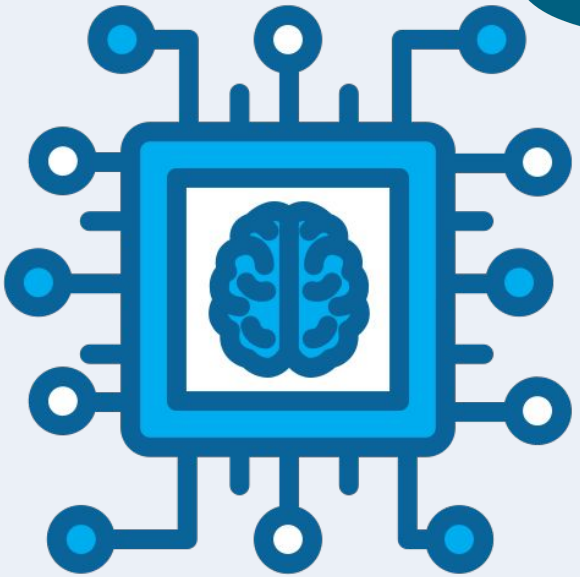
Input Features

Output Label



Tensor
BEYOND DIMENSIONS

Parameters



GPT-4 is estimated to have 500B+ parameters

Parameters are the internal values that a model learns during the training process to fit the data. Parameters affect how input features are transformed into output predictions.

The goal of training is to find the best parameters that have a minimum amount of loss/error.

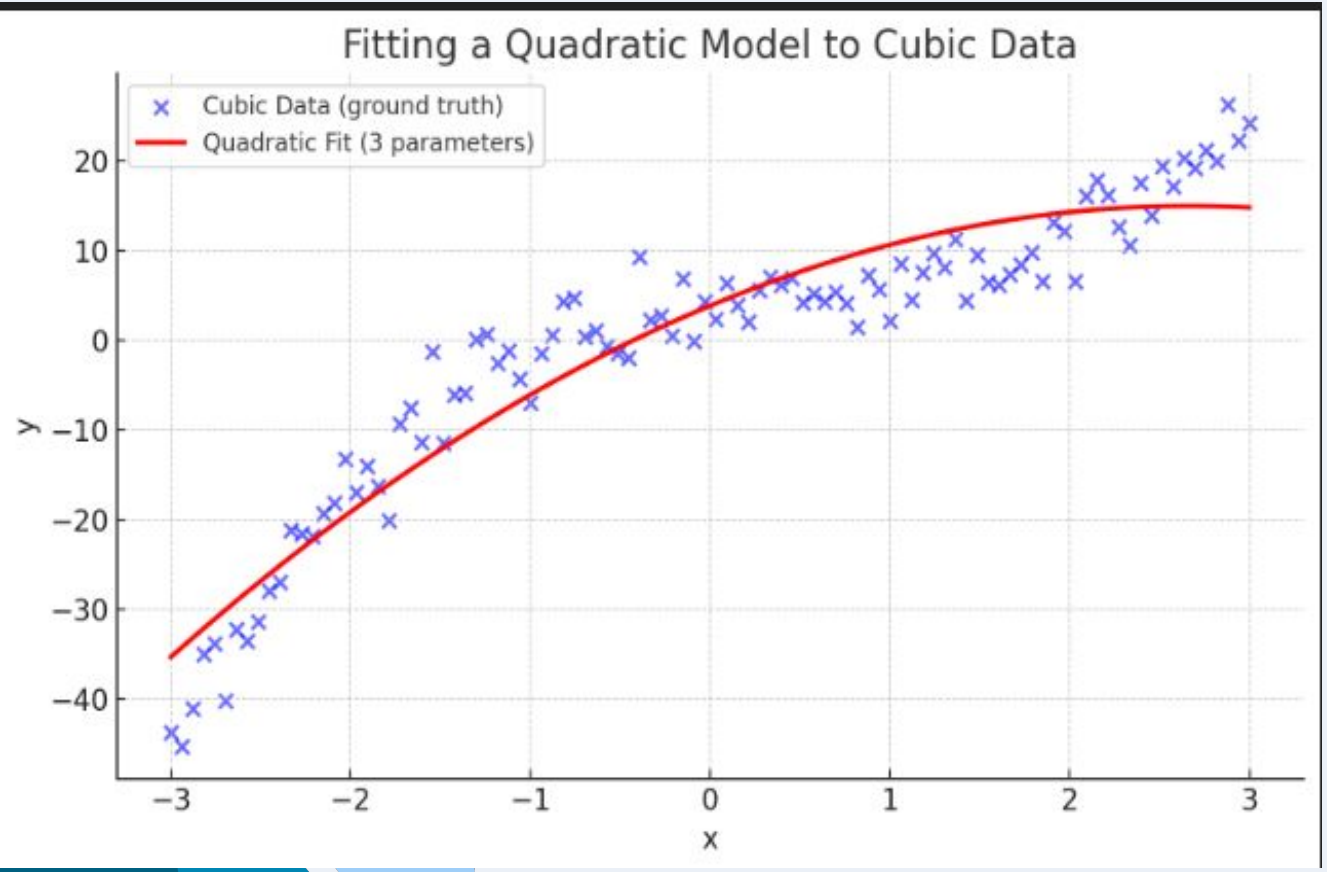
Tensor
BEYOND DIMENSIONS

How to decide the number of parameters ?

1. **Complexity of the problem**
   For a simpler problems like linear regression or binary classification, use few parameters, usually *number_of_features + 1*
   For complex problem like *Image Classification* or *Natural Language Processing*, use millions of parameter
2. **Larger data set usually require more parameters *(this may not be applicable for smaller models)***
3. **Avoid Overfitting/Underfitting**
   Too few parameters = model can't understand data (underfitting)
   Too many parameters = model can't generalize data (overfitting)

For our dataset (brightness Level x power consumed), we need 3 parameters to capture the data trend

Tensor
BEYOND DIMENSIONS



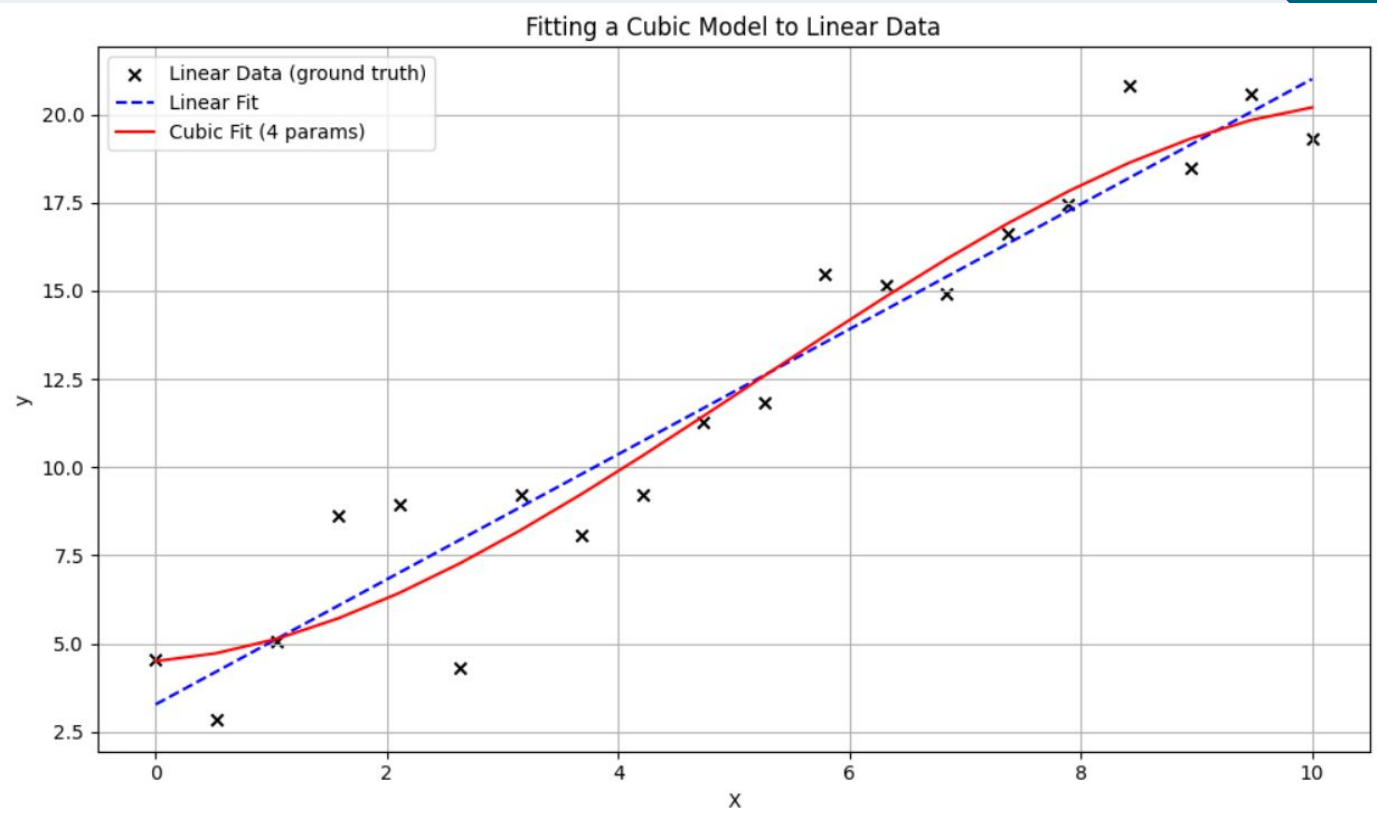
Cubic data needs 4 parameters as $y\_cubic = ax^3 + bx^2 + cx + d$

Here we have forced model to 3 parameters as
$y\_quad = ax^2 + bx + c$
so it has missed the cubic part of curve

Tensor
BEYOND DIMENSIONS

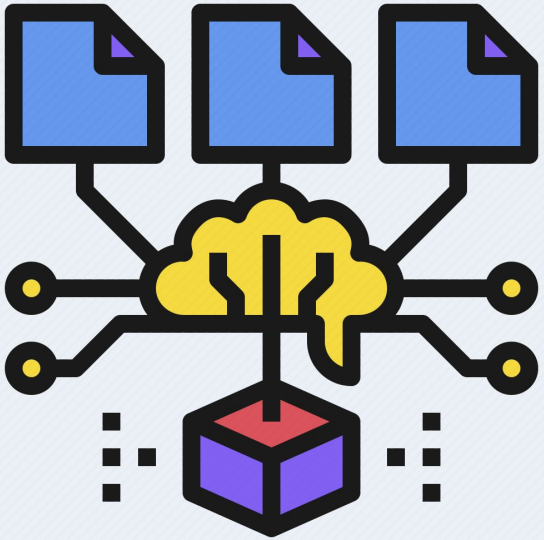
Linear data needs 2 parameters.
y = mx + b

Here we have forced model to 4 parameters so it has captured unnecessary trends in data



Tensor
BEYOND DIMENSIONS

Linear Combination



A linear combination is a mathematical way of combining parameters with input features to make predictions.

General form,

$$y = w_1 x_1 + w_2 x_2 + w_3 x_3 + \ldots + w_n x_n + b$$

Vector Form,

$$y = W^T + b$$

Mathematical combination in some ML models

1. **Linear Regression**

   $y = w_1 x_1 + w_2 x_2 + w_3 x_3 + \ldots + w_n x_n + b$

2. **Logistic Regression (binary classification)**

   $y = \sigma(w_1 x_1 + w_2 x_2 + w_3 x_3 + \ldots + w_n x_n + b)$

3. **Polynomial Regression (Generalized)**

   $y = (w_{11} x_1^{n} + w_{12} x_1^{n-1} + \ldots + w_{1n} x_1) + (w_{21} x_2^{n} + w_{22} x_2^{n-1} + \ldots + w_{2n} x_2) + \ldots\ldots + b$

4. **Exponential Regression**

   $y = e^{W1X1 + W2X2 + \ldots + WnXn + b}$

5. **Neural Network (Neuron)**

   $y = \Phi (w_1 x_1 + w_2 x_2 + w_3 x_3 + \ldots + w_n x_n + b)$

6. **Polynomial Regression (2nd degree)**

   $y = w_1 x^2 + w_2 x + b$

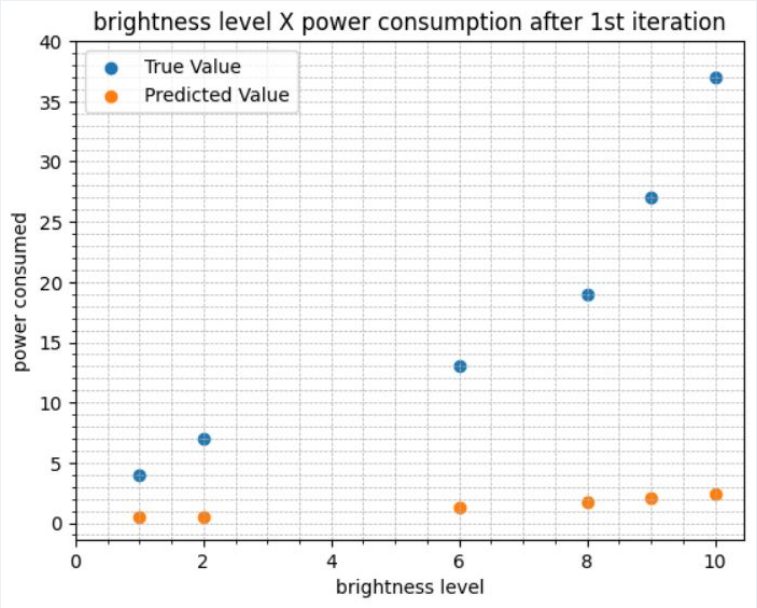
Predicting the output labels with help of linear combination

X	y_true	y_pred
1	4	0.465
2	7	0.56
6	13	1.24
8	19	1.76
9	27	2.065
10	37	2.4

$W_1 = 0.015, W_2 = 0.05, b = 0.4$

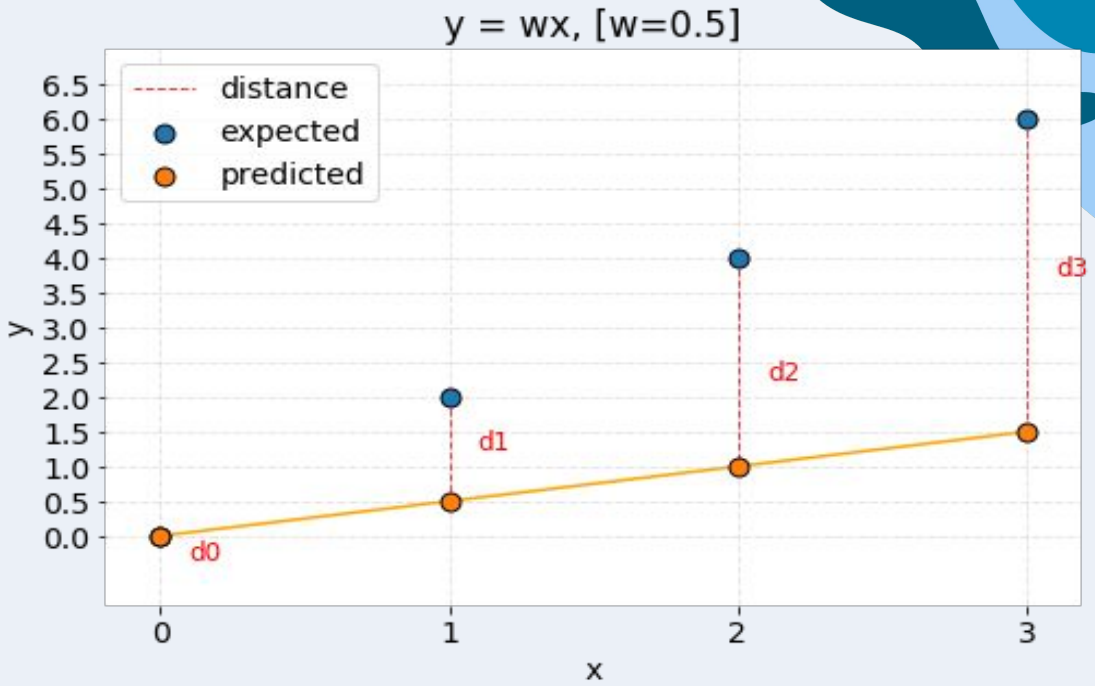
$y\_pred = W_1X^2 + W_2X + b$

$y_0 = 0.015 * 1^2 + 0.05 * 1 + 0.4$
$= 0.465$



Tensor
BEYOND DIMENSIONS

Cost Function



A cost function (also called a loss function) is a mathematical formula that tells you how wrong your model's predictions are. It measures the **difference between the predicted output and the actual output** (label) in a machine learning model.

Mean Squared Error (MSE)

What is error?

The difference between **“भएको कुरा”** and **“हुनु पर्ने कुरा”** is error.

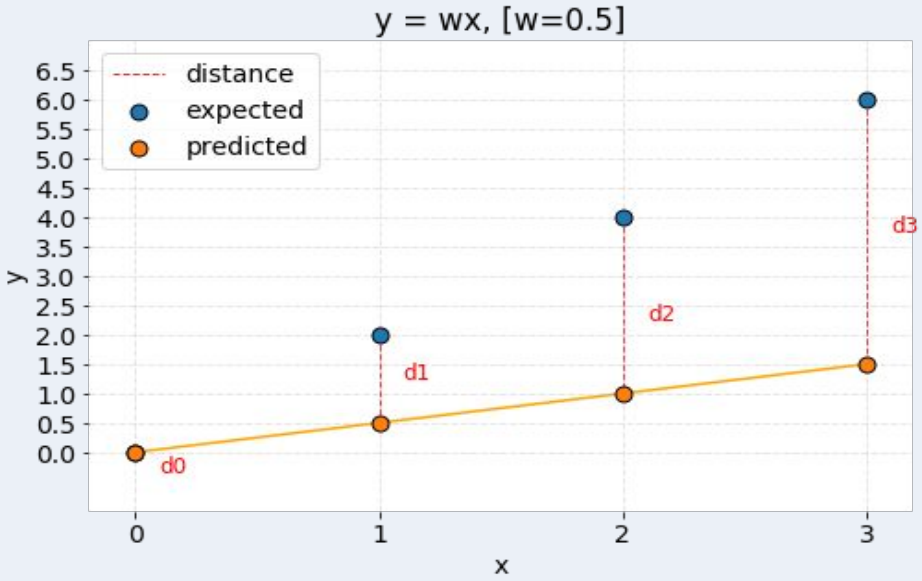
e (error) = भएको कुरा (obtained value) - हुनु पर्ने कुरा (true value)

MSE = mean of (e^2) for all data points

$$= (1 / n) \sum_{i=0}^{n} (y' - y)^2$$

In the figure beside,

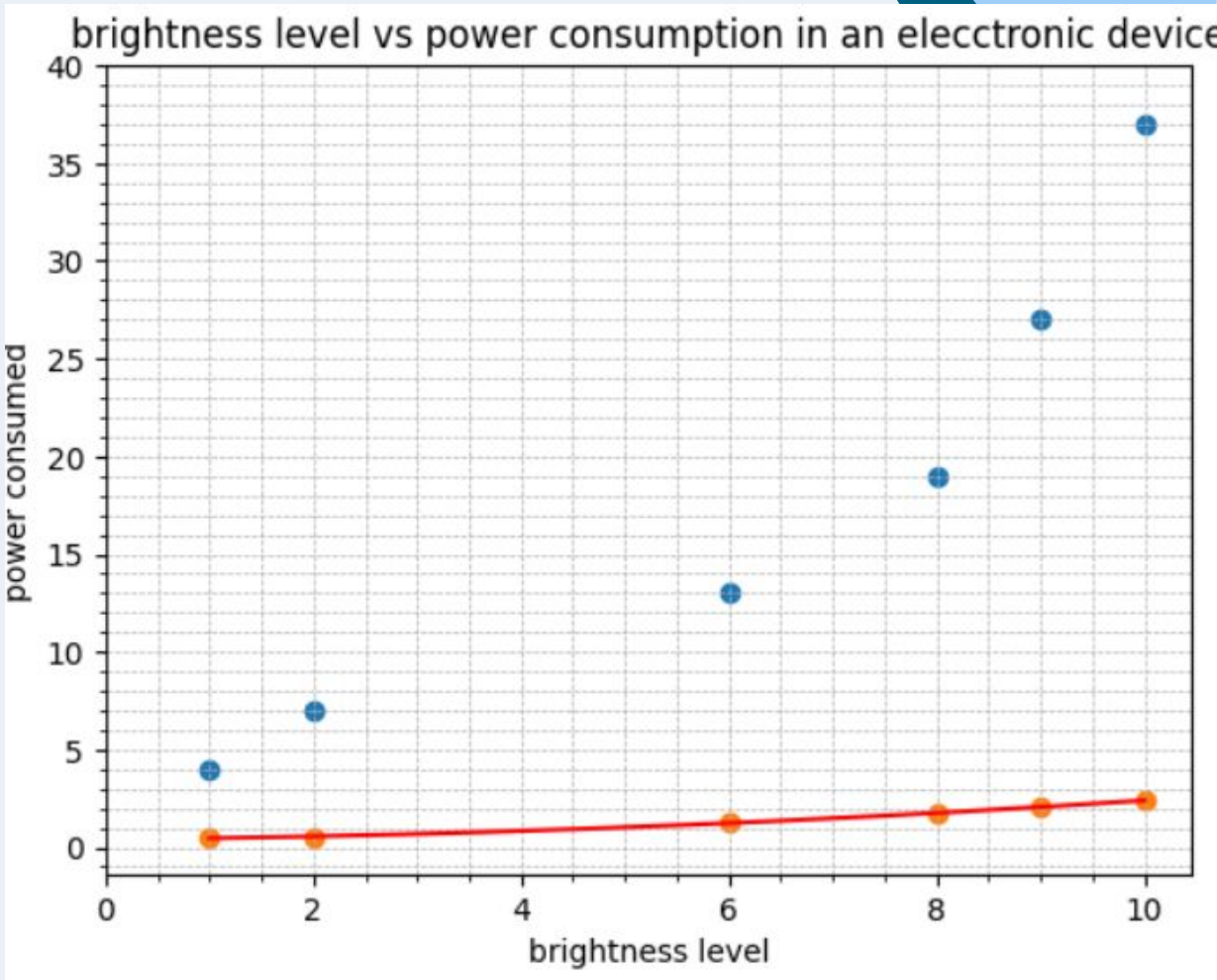
$$MSE = (d0^2 + d1^2 + d2^2 + d3^2) / 4$$
$$= (0 + (0.5 - 2)^2 + (1 - 4)^2 + (1.5 - 6)^2) / 4$$
$$= (1.5^2 + 3^2 + 4.5^2) / 4$$
$$= 7.875$$



Tensor
BEYOND DIMENSIONS

Number of data points (n) = 6
d0 = 1- 4 = -3 | $d0^2$ = 9
d1 = 1 - 7 = -6 | $d1^2$ = 36
d2 = 1 - 13 = -12 | $d2^2$ = 144
d3 = 2 - 19 = -17 | $d3^2$ = 289
d4 = 2 - 27 = -25 | $d4^2$ = 625
d5 = 3 - 37 = -34| $d5^2$ = 1156

Sum = 9 + 36 + 144 + 289 + 625 + 1156 = 2259
MSE = sum / n = 2259 / 6 = 376.5



Tensor
BEYOND DIMENSIONS

some other popular cost functions used

1. **Mean Absolute Error (MAE) >** Regression

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

2. **Binary Cross-Entropy (Log Loss) >** Binary Classification

$$\text{BCE} = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)]$$

3. **Categorical Cross-Entropy >** Multiclass Classification

$$\text{CCE} = -\sum_{i=1}^{n}\sum_{c=1}^{C} y_{i,c}\log(\hat{y}_{i,c})$$

4. **Hinge Loss >** Support Vector Machines

$$\text{Hinge Loss} = \sum_{i=1}^{n}\max(0, 1 - y_i\hat{y}_i)$$

5. **Huber Loss >** Regression

$$L_\delta(a) = \begin{cases} \frac{1}{2}a^2 & \text{for } |a| \le \delta, \\ \delta(|a| - \frac{1}{2}\delta) & \text{for } |a| > \delta, \end{cases}$$

Why do we use MSE for regression models?

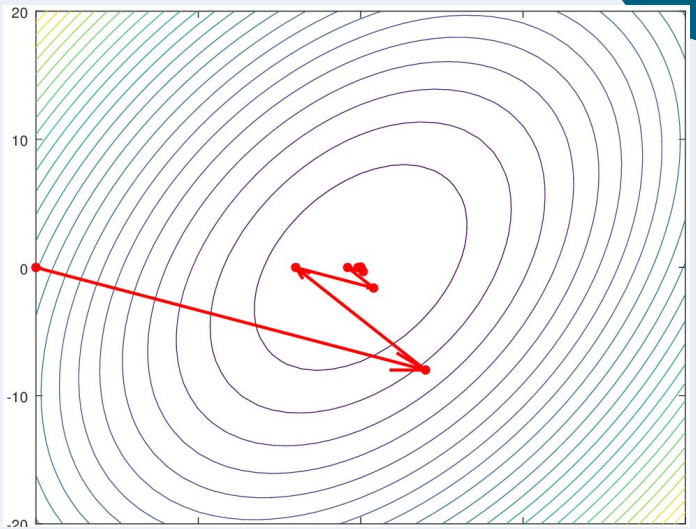
1. **Penalizes large errors:** Squaring the differences amplifies large errors, making the model more sensitive to significant mispredictions. ***In MSE, two mispredictions with a difference of 3 are far less than 1 misprediction with a difference of 6.***
2. **Smooth differentiability:** Since MSE involves squared terms, it results in a smooth gradient, making optimization easier for algorithms like gradient descent.
3. **Convex For Linear Models:** MSE produces convex cost surface for linear models, hence ensuring there is single global minimum

When to avoid MSE?

1. **If data contains outliers:** as it squares the error, the presence of a single outlier can dominate the cost function.
2. **Noisy Data:** MSE can amplify the noise
3. **Asymmetric Cost:** MSE treats positive and negative errors equally; if cost of underestimation and overestimation differs, then MSE is not suitable for, e.g., financial forecasting
4. **Data with varying scales :** If data has features with different scales, then MSE will be dominated by features with large ranges.

Tensor
BEYOND DIMENSIONS

Gradient Descent



Gradient descent is an optimization algorithm used in machine learning to minimize the cost function by iteratively adjusting parameters in the direction of the negative gradient, aiming to find the optimal set of parameters.

Tensor
BEYOND DIMENSIONS

Gradient Descent analogy

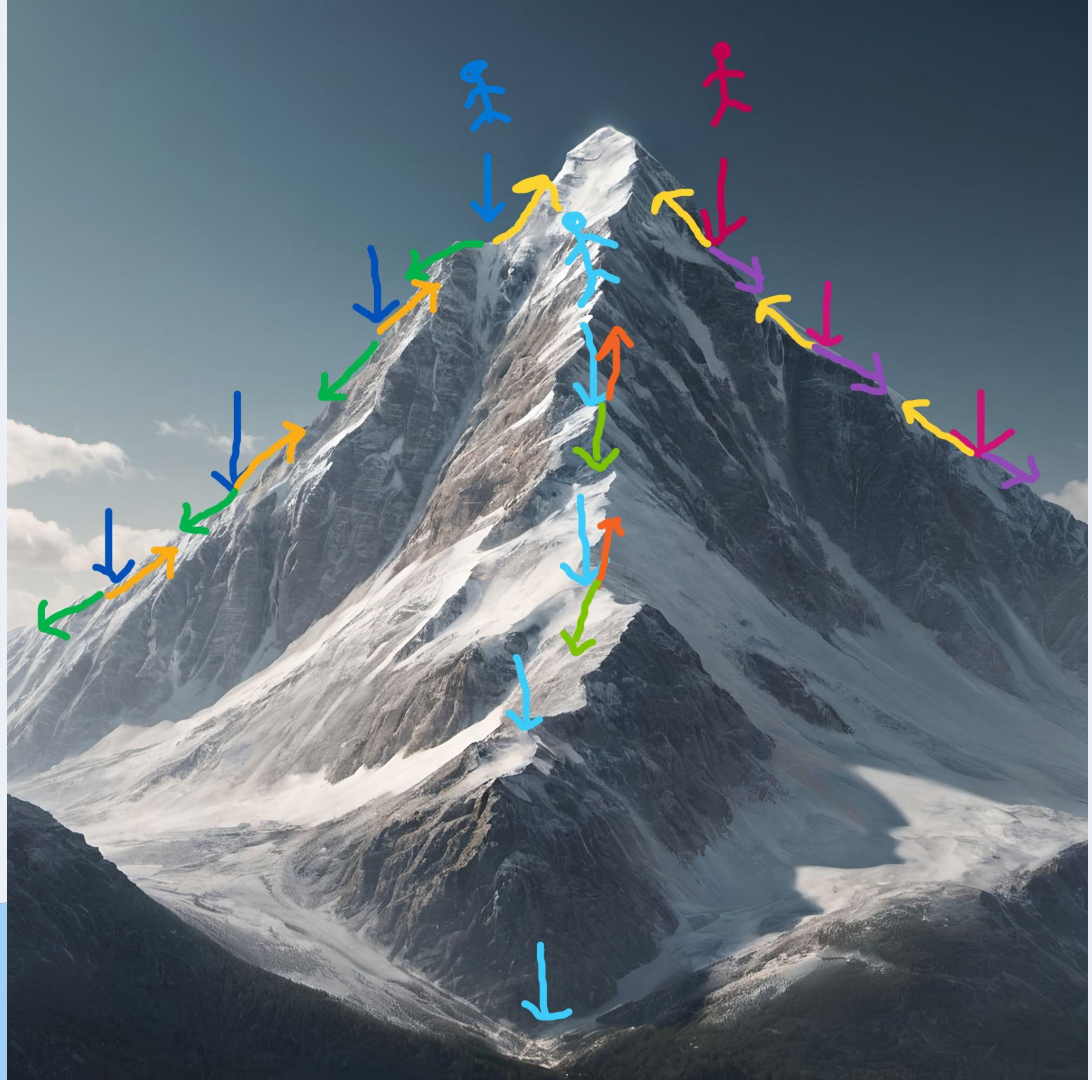
Imagine you are on a mountain trying to climb down to the valley below in the dark. You can't see the entire mountain, you can't see the valley, and you definitely can't see the fastest way down. So, what do you do?

You'd probably shuffle your feet around a bit to figure out which direction is most downhill at your current exact spot.

You're looking for the steepest immediate decline. Once you've identified the steepest downhill path, you'd take a careful, small step in that direction. You wouldn't jump off a cliff, so a small step allows you to reassess.

After that step, you'd again feel the ground to find the new steepest downhill direction from your new position and take another small step. In this way you can climb down the mountain to a valley.

Tensor
BEYOND DIMENSIONS



How does this algorithm work?

We have the cost function, $j(w, b) = 1 / n \sum (y' - y)^2$

Where $y' = w_1 x^2 + w_2 x + b$

In the function, x and y are constant input data. To find the optimal values for the weights w1, w2, and bias b, we calculate the partial derivatives of the loss function J(w,b) with respect to w1,w2 and b. This process, often called finding the gradient, helps us determine the optimal w and b.

Updating parameters, $Param = Param - \alpha \nabla J$

Here,

param : parameters of the model (in this case w, b)

α : Learning Rate

∇J : Partial derivative of the cost function wrt the parameter

Gradients of J(w,b) wrt w_1, w_2 and b

$$y' = w_1 x^2 + w_2 x + b$$

$$j(w_1, w_2, b) = \frac{1}{n} \sum_{n=1}^{n} (y' - y)^2$$

$$\frac{\partial j(w_1, w_2, b)}{\partial b} = \frac{\partial}{\partial b} \times \frac{1}{n} \sum_{i=1}^{n} (y - y')^2$$

$$= \frac{1}{n} \sum_{i=1}^{n} 2 \cdot (y - y') \times \frac{-\partial y'}{\partial b}$$

$$= \frac{2}{n} \sum_{i=1}^{n} (y - y') \times 1 \times \frac{-\partial (w_1 x^2 + w_2 x + b)}{\partial b}$$

$$= \frac{-2}{n} \sum_{i=1}^{n} (y - y') \times 1$$

$$\therefore \frac{\partial j(w_1, w_2, b)}{\partial b} = \frac{-2}{n} \sum_{i=1}^{n} (y - y')$$

$$\frac{\partial j(w_1, w_2, b)}{\partial w_1} = \frac{\partial}{\partial w_1} \times \frac{1}{n} \sum_{i=1}^{n} (y - y')^2$$

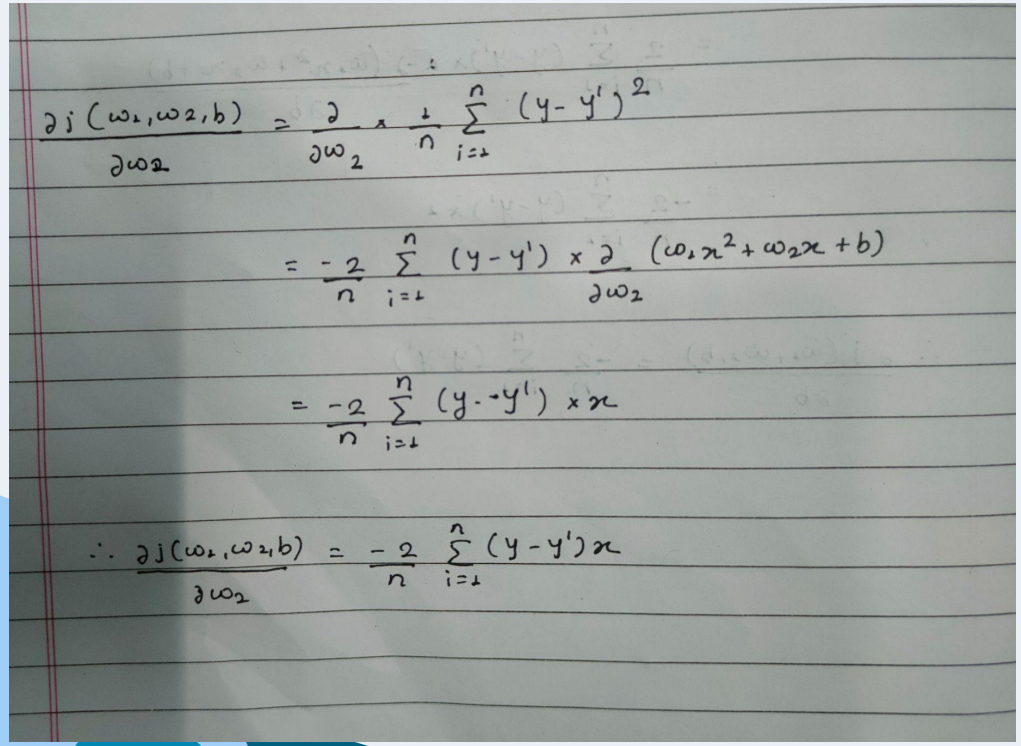
$$= \frac{1}{n} \sum_{i=1}^{n} 2 (y - y') \times \frac{\partial}{\partial w_1} (-y')$$

$$= \frac{-2}{n} \sum_{i=1}^{n} (y - y') \frac{\partial}{\partial w_1} (w_1 x^2 + w_2 x + b)$$

$$= \frac{-2}{n} \sum_{i=1}^{n} (y - y') x^2$$

$$\therefore \frac{\partial j(w_1, w_2, b)}{\partial w_1} = \frac{-2}{n} \sum_{i=1}^{n} (y - y') x^2$$

Updating the parameters w_1, w_2 and b



$$\frac{\partial j(w_1,w_2,b)}{\partial w_2} = \frac{\partial}{\partial w_2} \cdot \frac{1}{n} \sum_{i=1}^{n} (y - y')^2$$

$$= \frac{-2}{n} \sum_{i=1}^{n} (y - y') \times \frac{\partial}{\partial w_2} (w_1 x^2 + w_2 x + b)$$

$$= \frac{-2}{n} \sum_{i=1}^{n} (y - y') \times x$$

$$\therefore \frac{\partial j(w_1,w_2,b)}{\partial w_2} = \frac{-2}{n} \sum_{i=1}^{n} (y - y') x$$

Updating the parameters:

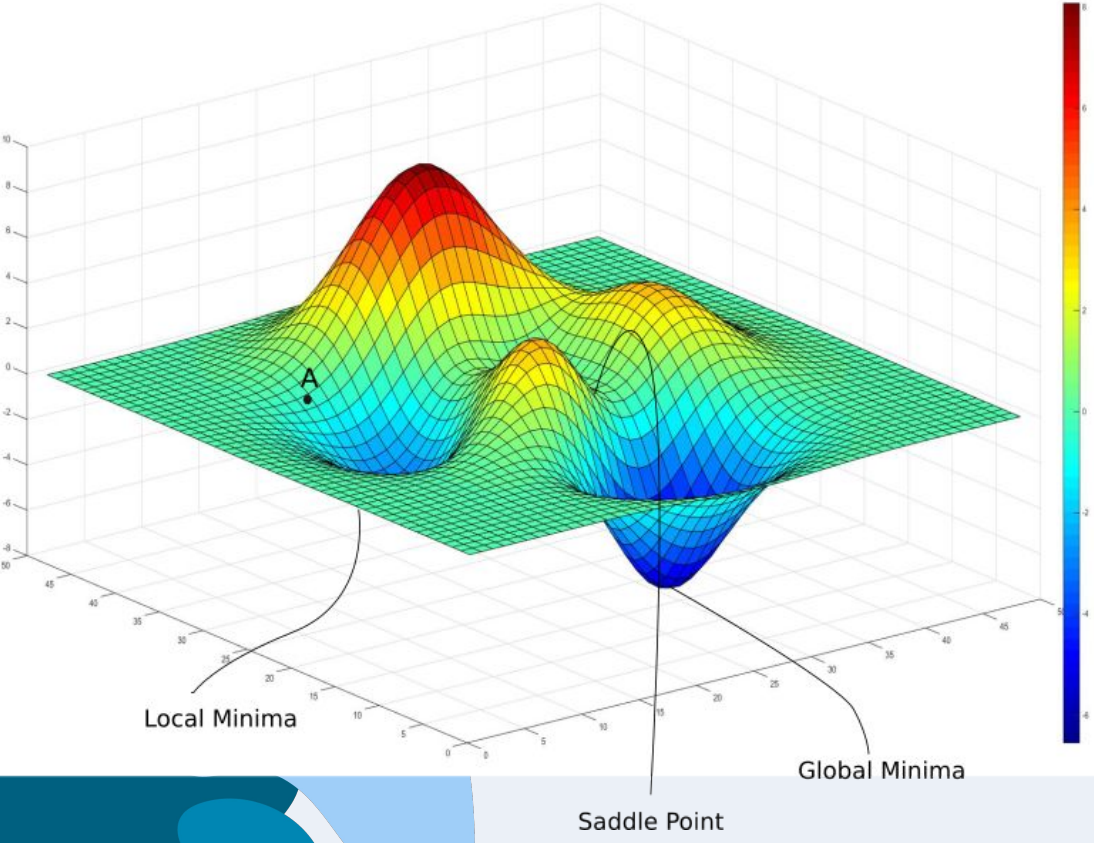
$w_1 = w_1 - \alpha\ (\partial j / \partial w_1)$

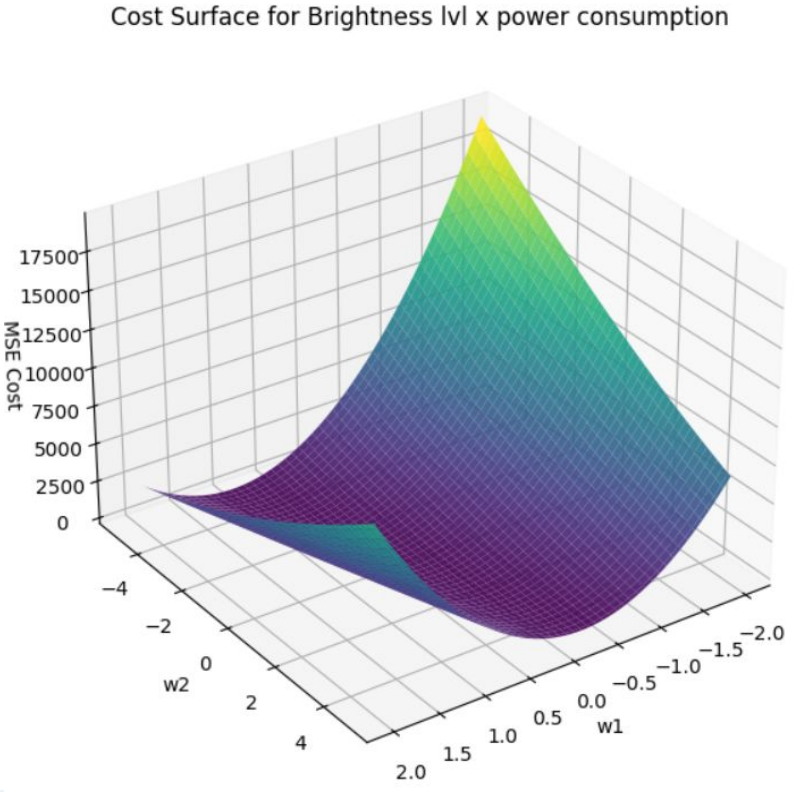
$w_2 = w_2 - \alpha\ (\partial j / \partial w_2)$

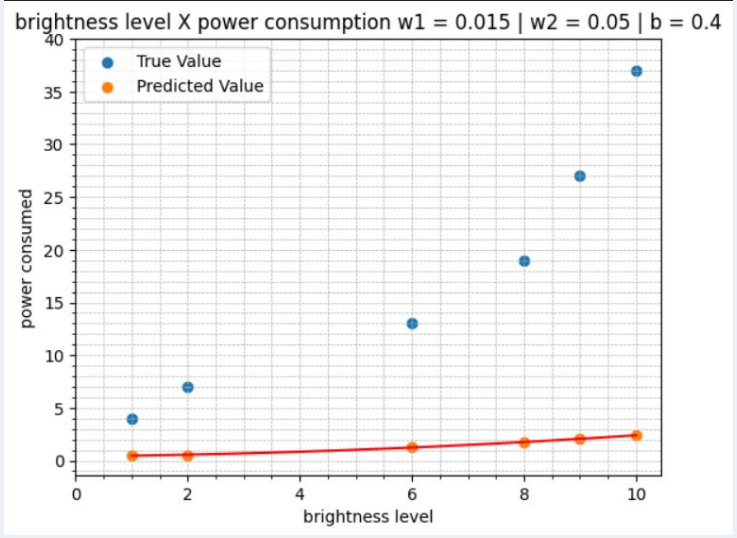
$b = b - \alpha\ (\partial j / \partial b)$

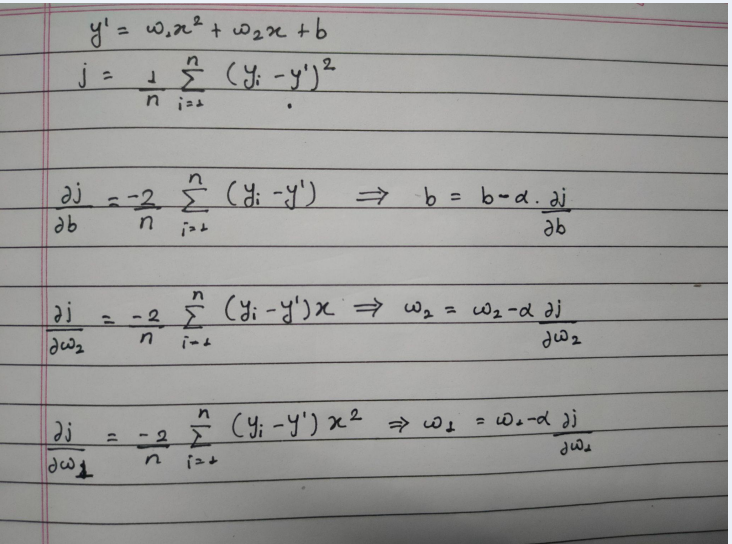
α is the learning rate. It is a hyperparameter that tells how big or small the step should be.

Understanding cost surfaces









y' = [1, 1, 1, 2, 2, 3]
y = [4, 7, 13, 19, 27, 37]
x = [1, 2, 6, 8, 9, 10]
MSE = 376.5, $\alpha = 0.0001$

for b,

$\sum (y - y') = (4 - 2) + (7 - 1) + (13 - 1) + (19 - 2) + (27 - 2) + (37 - 3)$
$= 2 + 6 + 12 + 17 + 25 + 34$
$= 96$

$\partial j / \partial b = (-2 / 6) * 96 = -32$
$b = 0.4 - (0.0001 * -32) = 0.403$

for w2,

$\sum (y - y')X = 2 * 1 + 6 * 2 + 12 * 6 + 17 * 8 + 25 * 9 + 34 * 10$
$= 787$

$\partial j / \partial b = (-2 / 6) * 787 = -262.333$
$w_2 = 0.05 - (0.0001 * -262.333) = 0.0762$

for w1,

$\sum (y - y')X^2 = 2 * 1 + 6 * 4 + 12 * 36 + 17 * 64 + 25 * 81 + 34 * 100$
$= 6971$

$\partial j / \partial b = (-2 / 6) * 6971 = -2323.667$
$w_1 = 0.015 - (0.0001 * -2323.667) = 0.2474$

Updating the model ($w_1 = 0.2474, w_2 = 0.07$

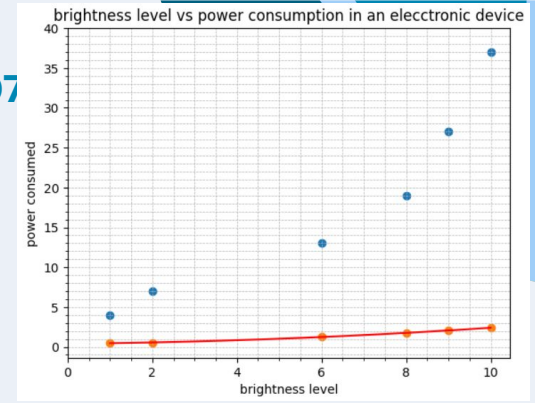
Tensor
BEYOND DIMENSIONS

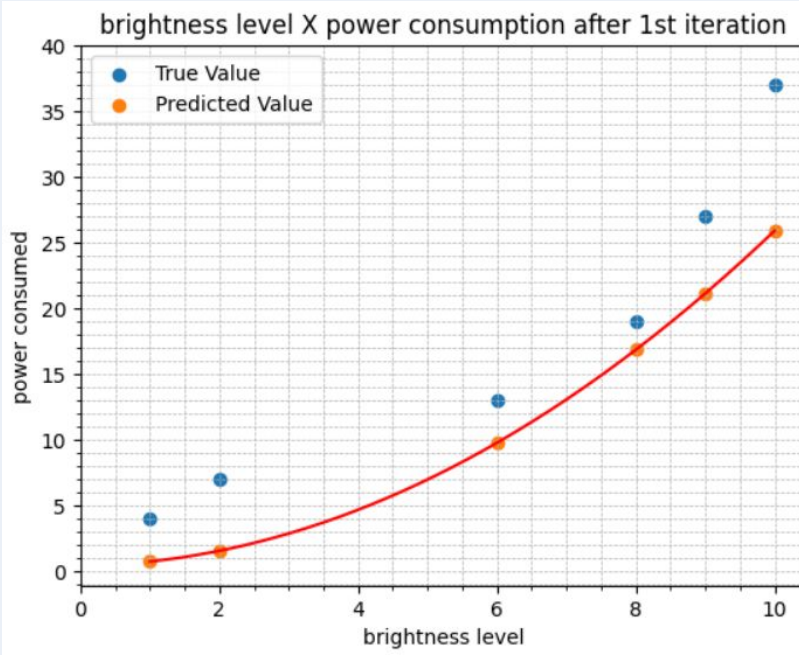
Brightness level	Power Consumed (true)	Power Consumed (predicted)
1	4	0.7266
2	7	1.545
6	13	9.7666
8	19	16.8462
9	27	21.1282
10	37	25.905

$y_0 = 0.2474 * 1^2 + 0.0762 * 1 + 0.403$
$= 0.7266$

$y_1 = 1.545$
$y_2 = 9.7666$
$y_3 = 16.8462$
$y_4 = 21.1282$
$y_5 = 25.905$

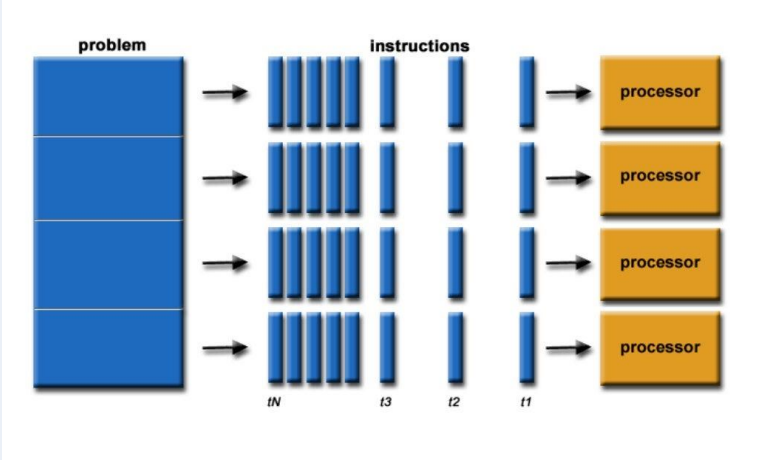
MSE = 35.2





Tensor
BEYOND DIMENSIONS

Vectorization



Vectorization is the process of converting operations that handle one element at a time into operations that process multiple elements simultaneously.

In libraries like NumPy, you can perform operations on entire arrays at a time rather than looping through each element.

Python Copy

```python
import numpy as np

# Without vectorization (using loops)
arr = np.array([1, 2, 3, 4, 5])
squared = []
for x in arr:
    squared.append(x ** 2)

# With vectorization
squared_vectorized = arr ** 2  # Fast
```

Tensor
BEYOND DIMENSIONS

Vectorization relies on a few important concepts :

1) Single Instruction Multiple Data (SIMD)

CPUs execute the same operation on multiple data points simultaneously.

2) Basic Linear Algebra Subprograms (BLAS)

It is a standardized collection of low-level routines that perform common linear algebra operations. It uses an HDL-like structure that knows about ***CPU cache, memory layout, and multithreading.*** They are written in low-level languages like C, Fortran, and Assembly.

3) Broadcasting

Broadcasting is the process that NumPy uses to perform operations on arrays of different shapes as if they had the same shape, without copying data.

GPUs

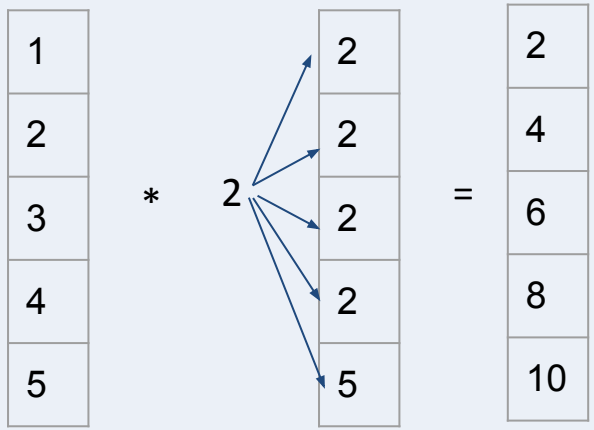
Modern libraries rely heavily on GPUs for vectorization. GPUs have many cores (physical and virtual), and it is built in SIMD architecture

Tensor
BEYOND DIMENSIONS

SIMD

1		2		3
2		4		6
3	+	6	=	9
4		8		12
5		10		15

Broadcasting



THANK YOU FOR YOUR ATTENTION!